

Skelpunktsfinder

Læremateriale — version 2 (opdateret med deployment, deling og avancerede UI-teknikker)

Dette er en opdateret udgave af lærematerialet. Den indeholder alt fra version 1 (vibe coding, Git, React, Leaflet, koordinatsystemer, sikker API-integration, GraphQL) samt en række nye emner, der kom til, efterhånden som appen voksede: deployment til eget domæne, deling af visninger via komprimerede links, og nogle mere avancerede UI- og browser-teknikker.

Indhold

1–13: Grundlæggende emner (uændret fra v1 — se tidligere udgave)

14. Deployment: statisk hosting vs. serverless functions

15. Undermapper og "base path" i Vite

16. URL'en som datalager: deep-linking og deling

17. Komprimering af data til URL'er

18. React Portals — når noget skal "undslippe" sin plads i koden

19. CSS stacking contexts — hvorfor z-index ikke altid virker

20. Responsivt tabel-layout med mange rækker

21. Geografiske forespørgsler i Leaflet (bounds.contains)

22. Filsystemer: store/små bogstaver på tværs af platforme

23. Afhængigheder og sikkerhed (npm audit)

24. git archive — hente en ren kopi af committet kode

Opdateret ordliste

1–13. Grundlæggende emner

De første 13 afsnit (vibe coding, VS Code/Node/npm, Git, Vite+React-opsætning, React-grundbegreber, Leaflet-kort, koordinatsystemer, sikker API-integration via proxy, GraphQL, projektstruktur, fejlfindingsmønstre, kommando-opslag og ordliste) er uændrede fra første udgave af dette dokument. De nye emner fra version 2 starter herunder.

14. Deployment: statisk hosting vs. serverless functions

En vigtig lære fra projektet: den lokale proxy i vite.config.js (som skjuler API-nøgler under udvikling) er kun en del af *udviklingsserveren* — den følger ikke automatisk med, når man bygger appen til en statisk hjemmeside. Almindelig delt webhosting (som One.com) kan vise statiske filer, men ikke køre en vedvarende Node.js-baggrundsproces.

Løsningen: to adskilte dele. Det visuelle (den byggede app) hostes statisk på ét sted; en lille "nøgle-skjuler"-funktion (en **serverless function**) hostes et andet sted (her: Vercel, gratis niveau) og har kun til opgave at modtage kald, tilføje API-nøglen usynligt, og sende videre.

```
// api/skelpunkter.js – kører på Vercel, ikke i browseren
export default async function handler(req, res) {
  const apiKey = process.env.DATAFORDELER_APIKEY
  // ... videresend kaldet med nøglen tilføjet
}
```

15. Undermapper og "base path" i Vite

Hvis en app skal ligge på dindomæne.dk/undermappe/ i stedet for på domænets forside, skal Vite fortælles det eksplicit — ellers bygges alle interne filstier (til JavaScript, CSS, ikoner) forkert, og siden vises blank.

```
export default defineConfig({
  base: '/undermappe/',
  ...
})
```

God fejlfindingsvane: åbn den byggede index.html (i dist-mappen) og tjek, at <script src="...">-stien rent faktisk indeholder undermappen, før man uploader.

16. URL'en som datalager: deep-linking og deling

En URL kan bære data med sig via forespørgselsparametre (det, der kommer efter ? i adressen). Det gør det muligt at "gemme" en tilstand — fx en liste af punkter, et valgt koordinatsystem, og kortets position — direkte i selve linket, uden nogen database. Åbner man linket igen, læses tilstanden ud af adressen og bruges til at genopbygge visningen.

```
https://side.dk/app/?cs=dktm3&view;=55.68,12.57,15&pts;=...
```

Vigtigt princip: del kun det, der ikke let kan hentes igen. I dette projekt deler vi *ikke* selve skelpunkterne/skellinjerne i linket — de kan altid hentes friskt fra Dataforsyningen igen. I stedet

auto-henter appen dem, når en delt visning åbnes. Det holder linket kort og giver altid opdaterede data.

17. Komprimering af data til URL'er

Browsere og servere har en praktisk grænse for, hvor lange URL'er/headers de accepterer (typisk 8.000–16.000 tegn) — med mange punkter i et link kan man ramme den grænse (fejlen 431 `Request Header Fields Too Large`).

Løsningen er at komprimere data, før den lægges i URL'en, med et lille bibliotek som **lz-string**. Tal og gentagne mønstre (som koordinater ofte er) komprimerer særligt godt — i dette projekt gav det 50-70% mindre tekst, mest markant ved mange punkter.

```
import LZString from 'lz-string'
const kort = LZString.compressToEncodedURIComponent(langTekst)
const langTekst = LZString.decompressFromEncodedURIComponent(kort)
```

■ Ved 300 punkter gik teksten fra ca. 9.300 tegn til ca. 2.900 — en forskel der afgør, om linket overhovedet virker.

18. React Portals — når noget skal "undslippe" sin plads i koden

Normalt tegnes en Reacts-komponent nøjagtigt der, hvor den står i koden — som barn af sine forældre-komponenter. Nogle gange (fx en informationsboks, der skal ligge *ovenpå* alt andet, uanset hvor i koden den kaldes fra) er det praktisk at bryde med det. En **portal** lader en komponent tegnes et helt andet sted i selve HTML-dokumentet (typisk direkte i `document.body`), selvom den logisk stadig "bor" samme sted i koden.

```
import { createPortal } from 'react-dom'

{open && createPortal(
  <div style={{ position: 'fixed', top, left }}>...</div>,
  document.body
)}
```

19. CSS stacking contexts — hvorfor z-index ikke altid virker

En klassisk CSS-fælde: at sætte `z-index` højt løser ikke altid, at noget ligger "under" noget andet. `z-index` sammenlignes kun *inden for samme lag-kontekst* — hvis en forælder-boks selv opretter sin egen kontekst (fx via `position + z-index`, eller `transform`), kan intet `z-index`-tal inde i den boks vinde over noget uden for den, uanset hvor højt tallet er.

I dette projekt viste det sig som en info-boks, der forblev skjult bag et Leaflet-kort, selvom `z-index` blev sat meget højt. Den robuste løsning var en portal (se ovenfor) — send elementet helt uden om problemet, i stedet for at forsøge at "vinde" lag-kampen.

20. Responsivt tabel-layout med mange rækker

Med en liste, der kan vokse sig meget lang (hundredvis af rækker), bliver én lang tabel hurtigt uoverskuelig. Løsningen: del listen op i flere kolonner, med et loft for, hvor mange kolonner der må bruges — og fordel derefter rækkerne så jævnt som muligt på tværs af dem, i stedet for at proppe den første kolonne helt fuld først.

```
function splitIntoColumns(items, maxPerColumn, maxColumns) {  
  const numColumns = Math.min(maxColumns, Math.ceil(items.length / maxPerColumn))  
  const baseSize = Math.floor(items.length / numColumns)  
  const remainder = items.length % numColumns  
  // ... fordel resten på de(n) første kolonne(r)  
}
```

Vigtig detalje: når man deler en liste op visuelt, skal hvert element stadig "huske" sit oprindelige indeks i den fulde liste (til redigering/sletning) — ikke kun sit indeks inden for sin egen kolonne.

21. Geografiske forespørgsler i Leaflet (bounds.contains)

Leaflet-kortobjektet kan fortælle, hvilket geografisk område der er synligt lige nu (`map.getBounds()`), og det resulterende "bounds"-objekt kan svare på, om et givent koordinat ligger inden for det synlige udsnit (`bounds.contains([lat, lng])`). Det er nyttigt til fx kun at handle på data, der rent faktisk er synlig for brugeren lige nu — i modsætning til alt data, der er hentet, men måske ligger uden for det aktuelle udsnit.

22. Filsystemer: store/små bogstaver på tværs af platforme

Windows og macOS er som udgangspunkt *ligeglade* med store/små bogstaver i filnavne — `InfoButton.jsx` og `infobutton.jsx` regnes for samme fil. Linux (som mange servere og CI/CD-systemer kører på) er derimod *kræsen* — de er to forskellige filer. Kode, der importerer en fil med "forkert" store/små bogstaver, kan derfor virke fint på en udviklers egen maskine, men fejle et andet sted.

■■ God vane: hold filnavne og import-stier konsekvente i store/små bogstaver, selv når ens eget system ikke kræver det.

23. Afhængigheder og sikkerhed (npm audit)

Når man installerer et nyt bibliotek, tjekker npm automatisk, om nogen af projektets afhængigheder (direkte eller indirekte) har kendte sikkerhedshuller, og advarer med en sværhedsgrad ("high", "moderate" osv.).

```
npm audit # se detaljer om fundne sårbarheder  
npm audit fix # opdater automatisk til rettede versioner, hvor muligt
```

Vigtigt at vide: en "high severity"-advarsel betyder ikke automatisk, at ens egen app er i fare — det afhænger af, *hvordan* (og om) den sårbare funktion overhovedet bruges i praksis. Det er værd at læse selve beskrivelsen af sårbarheden, ikke kun overskriften, før man vurderer risikoen.

24. git archive — hente en ren kopi af committet kode

Når man er i tvivl om, hvorvidt lokale filer stemmer overens med det, der faktisk er gemt i Git, er `git archive` en pålidelig måde at hente en "ren" kopi på — den pakker *kun* de filer, der er committet, ignorerer automatisk alt i `.gitignore`, og rører aldrig ved eventuelle ugemte ændringer på disken.

```
git archive -o mit-projekt.zip HEAD
```

Det var præcis den kommando, der løste en situation i dette projekt, hvor gentagne fil-genskabelser (på tværs af flere samtaler) var kommet ud af trit med den faktiske, committede kode.

Opdateret ordliste

Deep-linking — At kode en tilstand direkte ind i en URL, så et link alene kan genskabe en bestemt visning.

Komprimering — At omkode data til en tættere pakket form, for at spare plads (fx i en URL).

Portal (React) — En mekanisme der lader en komponent tegnes et andet sted i HTML'en, end hvor den logisk står i koden.

Stacking context — Et lag-hierarki i CSS — z-index sammenlignes kun inden for samme kontekst, ikke globalt.

Serverless function — Et lille stykke serverkode, der kun kører, når det kaldes — kræver ingen vedvarende server.

Base path — Den undermappe-sti, en app er bygget til at ligge under, i stedet for domænets rod.

Case-sensitivity — Om store/små bogstaver regnes som forskellige (Linux) eller ej (Windows/macOS) i filnavne.